

Kleene's recursion theorem

In computability theory, **Kleene's recursion theorems** are a pair of fundamental results about the application of computable functions to their own descriptions. The theorems were first proved by Stephen Kleene in 1938^[1] and appear in his 1952 book *Introduction to Metamathematics*.^[2] A related theorem, which constructs fixed points of a computable function, is known as **Rogers's theorem** and is due to Hartley Rogers, Jr.^[3]

The recursion theorems can be applied to construct fixed points of certain operations on computable functions, to generate quines, and to construct functions defined via recursive definitions.

Notation

The statement of the theorems refers to an admissible numbering φ of the partial recursive functions, such that the function corresponding to index e is φ_e .

If F and G are partial functions on the natural numbers, the notation $F \simeq G$ indicates that, for each n , either $F(n)$ and $G(n)$ are both defined and are equal, or else $F(n)$ and $G(n)$ are both undefined.

Rogers's fixed-point theorem

Given a function F , a **fixed point** of F is an index e such that $\varphi_e \simeq \varphi_{F(e)}$. Note that the comparison of in- and outputs here is not in terms of numerical values, but in terms of their associated functions.

Rogers describes the following result as "a simpler version" of Kleene's (second) recursion theorem.^[4]

Rogers's fixed-point theorem—If F is a total computable function, it has a fixed point in the above sense.

This essentially means that if we apply an effective transformation to programs (say, replace instructions such as successor, jump, remove lines), there will always be a program whose behaviour is not altered by the transformation. This theorem can therefore be interpreted in the following manner: "given any effective procedure to transform programs, there is always a program that, when modified by the procedure, does exactly what it did before", or: "it's impossible to write a program that changes the extensional behaviour of all programs".

Proof of the fixed-point theorem

The proof uses a particular total computable function h , defined as follows. Given a natural number x , the function h outputs the index of the partial computable function that performs the following computation:

Given an input y , first attempt to compute $\varphi_x(x)$. If that computation returns an output e , then compute $\varphi_e(y)$ and return its value, if any.

Thus, for all indices x of partial computable functions, if $\varphi_x(x)$ is defined, then $\varphi_{h(x)} \simeq \varphi_{\varphi_x(x)}$. If $\varphi_x(x)$ is not defined, then $\varphi_{h(x)}$ is a function that is nowhere defined. The function h can be constructed from the partial computable function $g(x, y)$ described above and the s-m-n theorem: for each x , $h(x)$ is the index of a program which computes the function $y \mapsto g(x, y)$.

To complete the proof, let F be any total computable function, and construct h as above. Let e be an index of the composition $F \circ h$, which is a total computable function. Then $\varphi_{h(e)} \simeq \varphi_{\varphi_e}$ by the definition of h . But, because e is an index of $F \circ h$, $\varphi_e(e) = (F \circ h)(e)$, and thus $\varphi_{\varphi_e(e)} \simeq \varphi_{F(h(e))}$. By the transitivity of $\simeq$, this means $\varphi_{h(e)} \simeq \varphi_{F(h(e))}$. Hence $\varphi_n \simeq \varphi_{F(n)}$ for $n = h(e)$.

This proof is a construction of a partial recursive function which implements the Y combinator.

Fixed-point-free functions

A function F such that $\varphi_e \not\simeq \varphi_{F(e)}$ for all e is called **fixed-point free**. The fixed-point theorem shows that no total computable function is fixed-point free, but there are many non-computable fixed-point-free functions. **Arslanov's completeness criterion** states that the only recursively enumerable Turing degree that computes a fixed-point-free function is $0'$, the degree of the halting problem.^[5]

Kleene's second recursion theorem

The second recursion theorem is a generalization of Rogers's theorem with a second input in the function. One informal interpretation of the second recursion theorem is that it is possible to construct self-referential programs; see "Application to quines" below.

The second recursion theorem. For any partial recursive function $Q(x, y)$ there is an index p such that $\varphi_p \simeq \lambda y. Q(p, y)$.

The theorem can be proved from Rogers's theorem by letting $F(p)$ be a function such that $\varphi_{F(p)}(y) = Q(p, y)$ (a construction described by the S-m-n theorem). One can then verify that a fixed-point of this F is an index p as required. The theorem is constructive in the sense that a fixed computable function maps an index for Q into the index p .

Comparison to Rogers's theorem

Kleene's second recursion theorem and Rogers's theorem can both be proved, rather simply, from each other.^[6] However, a direct proof of Kleene's theorem^[7] does not make use of a universal program, which means that the theorem holds for certain subrecursive programming systems that do not have a universal program.

Application to quines

A classic example using the second recursion theorem is the function $Q(x, y) = x$. The corresponding index p in this case yields a computable function that outputs its own index when applied to any value.^[8] When expressed as computer programs, such indices are known as **quines**.

The following example in Lisp illustrates how the p in the corollary can be effectively produced from the function Q . The function `s11` in the code is the function of that name produced by the S-m-n theorem.

Q can be changed to any two-argument function.

```
(setq Q '(lambda (x y) x))
(setq s11 '(lambda (f x) (list 'lambda '(y) (list f x 'y))))
(setq n (list 'lambda '(x y) (list Q (list s11 'x 'x) 'y)))
(setq p (eval (list s11 n n)))
```

The results of the following expressions should be the same. $\varphi_p(\text{nil})$


```
(eval (list p nil))
```

$Q(p, \text{nil})$

```
(eval (list Q p nil))
```

Application to elimination of recursion

Suppose that g and h are total computable functions that are used in a recursive definition for a function f :

$$f(0, y) \simeq g(y),$$

$$f(x + 1, y) \simeq h(f(x, y), x, y),$$

The second recursion theorem can be used to show that such equations define a computable function, where the notion of computability does not have to allow, *prima facie*, for recursive definitions (for example, it may be defined by μ -recursion, or by Turing machines). This recursive definition can be converted into a computable function $\varphi_F(e, x, y)$ that assumes e is an index to itself, to simulate recursion:

$$\varphi_F(e, 0, y) \simeq g(y),$$

$$\varphi_F(e, x + 1, y) \simeq h(\varphi_e(x, y), x, y).$$

The recursion theorem establishes the existence of a computable function φ_f such that $\varphi_f(x, y) \simeq \varphi_F(f, x, y)$. Thus f satisfies the given recursive definition.

Reflexive programming

Reflexive, or reflective, programming refers to the usage of self-reference in programs. Jones presents a view of the second recursion theorem based on a reflexive language.^[9] It is shown that the reflexive language defined is not stronger than a language without reflection (because an interpreter for the reflexive language can be implemented without using reflection); then, it is shown that the recursion theorem is almost trivial in the reflexive language.

The first recursion theorem

While the second recursion theorem is about fixed points of computable functions, the first recursion theorem is related to fixed points determined by enumeration operators, which are a computable analogue of inductive definitions. An **enumeration operator** is a set of pairs (A, n) where A is a code for a finite set of numbers and n is a single natural number. Often, n will be viewed as a code for an ordered pair of natural numbers, particularly when functions are defined via enumeration operators. Enumeration operators are of central importance in the study of enumeration reducibility.

Each enumeration operator Φ determines a function from sets of naturals to sets of naturals given by

$$\Phi(X) = \{n \mid \exists A \subseteq X[(A, n) \in \Phi]\}.$$

A **recursive operator** is an enumeration operator that, when given the graph of a partial recursive function, always returns the graph of a partial recursive function.

A fixed point of an enumeration operator Φ is a set F such that $\Phi(F) = F$. The first enumeration theorem shows that fixed points can be effectively obtained if the enumeration operator itself is computable.

First recursion theorem. The following statements hold.

1. For any computable enumeration operator Φ there is a recursively enumerable set F such that $\Phi(F) = F$ and F is the smallest set with this property.
2. For any recursive operator Ψ there is a partial computable function φ such that $\Psi(\varphi) = \varphi$ and φ is the smallest partial computable function with this property.

The first recursion theorem is also called Fixed point theorem (of recursion theory).^[10] There is also a definition which can be applied to recursive functionals as follows:

Let $\Phi : \mathbb{F}(\mathbb{N}^k) \rightarrow (\mathbb{N}^k)$ be a recursive functional. Then Φ has a least fixed point $f_\Phi : \mathbb{N}^k \rightarrow \mathbb{N}$ which is computable i.e.

1) $\Phi(f_\Phi) = f_\Phi$

2) $\forall g \in \mathbb{F}(\mathbb{N}^k)$ such that $\Phi(g) = g$ it holds that $f_\Phi \subseteq g$

3) f_Φ is computable

Example

Like the second recursion theorem, the first recursion theorem can be used to obtain functions satisfying systems of recursion equations. To apply the first recursion theorem, the recursion equations must first be recast as a recursive operator.

Consider the recursion equations for the factorial function f :

$$\begin{aligned} f(0) &= 1 \\ f(n+1) &= (n+1) \cdot f(n) \end{aligned}$$

The corresponding recursive operator Φ will have information that tells how to get to the next value of f from the previous value. However, the recursive operator will actually define the graph of f . First, Φ will contain the pair $(\emptyset, (0, 1))$. This indicates that $f(0)$ is unequivocally 1, and thus the pair $(0, 1)$ is in the graph of f .

Next, for each n and m , Φ will contain the pair $(\{(n, m)\}, (n+1, (n+1) \cdot m))$. This indicates that, if $f(n)$ is m , then $f(n+1)$ is $(n+1)m$, so that the pair $(n+1, (n+1)m)$ is in the graph of f . Unlike the base case $f(0) = 1$, the recursive operator requires some information about $f(n)$ before it defines a value of $f(n+1)$.

The first recursion theorem (in particular, part 1) states that there is a set F such that $\Phi(F) = F$. The set F will consist entirely of ordered pairs of natural numbers, and will be the graph of the factorial function f , as desired.

The restriction to recursion equations that can be recast as recursive operators ensures that the recursion equations actually define a least fixed point. For example, consider the set of recursion equations:

$$\begin{aligned} g(0) &= 1 \\ g(n+1) &= 1 \\ g(2n) &= 0 \end{aligned}$$

There is no function g satisfying these equations, because they imply $g(2) = 1$ and also imply $g(2) = 0$. Thus there is no fixed point g satisfying these recursion equations. It is possible to make an enumeration operator corresponding to these equations, but it will not be a recursive operator.

Proof sketch for the first recursion theorem

The proof of part 1 of the first recursion theorem is obtained by iterating the enumeration operator Φ beginning with the empty set. First, a sequence F_k is constructed, for $k = 0, 1, \dots$. Let F_0 be the empty set. Proceeding inductively, for each k , let F_{k+1} be $F_k \cup \Phi(F_k)$. Finally, F is taken to be $\bigcup F_k$. The remainder of the proof consists of a verification that F is recursively enumerable and is the least fixed point of Φ . The sequence F_k used in this proof corresponds to the Kleene chain in the proof of the Kleene fixed-point theorem.

The second part of the first recursion theorem follows from the first part. The assumption that Φ is a recursive operator is used to show that the fixed point of Φ is the graph of a partial function. The key point is that if the fixed point F is not the graph of a function, then there is some k such that F_k is not the graph of a function.

Comparison to the second recursion theorem

Compared to the second recursion theorem, the first recursion theorem produces a stronger conclusion but only when narrower hypotheses are satisfied. Rogers uses the term **weak recursion theorem** for the first recursion theorem and **strong recursion theorem** for the second recursion theorem.^[3]

One difference between the first and second recursion theorems is that the fixed points obtained by the first recursion theorem are guaranteed to be least fixed points, while those obtained from the second recursion theorem may not be least fixed points.

A second difference is that the first recursion theorem only applies to systems of equations that can be recast as recursive operators. This restriction is similar to the restriction to continuous operators in the Kleene fixed-point theorem of order theory. The second recursion theorem can be applied to any total recursive function.

Generalized theorem

In the context of his theory of numberings, Ershov showed that Kleene's recursion theorem holds for any precomplete numbering.^[11] A Gödel numbering is a precomplete numbering on the set of computable functions so the generalized theorem yields the Kleene recursion theorem as a special case.^[12]

Given a precomplete numbering ν , then for any partial computable function f with two parameters there exists a total computable function t with one parameter such that

$$\forall n \in \mathbb{N} : \nu \circ f(n, t(n)) = \nu \circ t(n).$$

See also

- Denotational semantics, where another least fixed point theorem is used for the same purpose as the first recursion theorem.
- Fixed-point combinators, which are used in lambda calculus for the same purpose as the first recursion theorem.
- Diagonal lemma a closely related result in mathematical logic.

References

- Ershov, Yuri L. (1999). "Part 4: Mathematics and Computability Theory. 14. Theory of numbering". In Griffor, Edward R. (ed.). *Handbook of Computability Theory* (<https://books.google.com/books?id=KqeXZ4pPd5QC>). Studies in logic and the foundations of mathematics. Vol. 140.

Amsterdam: Elsevier. pp. 473–503. ISBN 9780444898821. OCLC 162130533 (<https://search.worldcat.org/oclc/162130533>). Retrieved 6 May 2020.

- Jones, Neil D. (1997). *Computability and complexity: From a Programming Perspective*. Cambridge, Massachusetts: MIT Press. ISBN 9780262100649. OCLC 981293265 (<https://search.worldcat.org/oclc/981293265>).
- Kleene, Stephen C. (1952). *Introduction to Metamathematics* (<https://archive.org/details/BibliothecaMathematicaStephenColeKleeneIntroductionToMetamathematicsWoltersNoordhoffPublishing1971>). Bibliotheca Mathematica. North-Holland Publishing. ISBN 9780720421033. OCLC 459805591 (<https://search.worldcat.org/oclc/459805591>). Retrieved 6 May 2020.
- Rogers, Hartley (1967). *Theory of recursive functions and effective computability* (<https://archive.org/details/theoryofrecursiv00roge>). Cambridge, Massachusetts: MIT Press. ISBN 9780262680523. OCLC 933975989 (<https://search.worldcat.org/oclc/933975989>). Retrieved 6 May 2020.

Footnotes

1. Kleene, Stephen C. (1938). "On notation for ordinal numbers" (http://www.thatmarcusfamily.org/philosophy/Course_Websites/Readings/Kleene%20-%20Ordinals.pdf) (PDF). *Journal of Symbolic Logic*. **3** (4): 150–155. doi:10.2307/2267778 (<https://doi.org/10.2307%2F2267778>). ISSN 0022-4812 (<https://search.worldcat.org/issn/0022-4812>). JSTOR 2267778 (<https://www.jstor.org/stable/2267778>). S2CID 34314018 (<https://api.semanticscholar.org/CorpusID:34314018>). Retrieved 6 May 2020.
2. Kleene 1952.
3. Rogers 1967.
4. Rogers 1967, §11.2.
5. Soare, R.I. (1987). *Recursively Enumerable Sets and Degrees: A Study of Computable Functions and Computably Generated Sets*. Perspectives in Mathematical Logic. Berlin and New York City: Springer-Verlag. p. 88. ISBN 9780387152998. OCLC 318368332 (<https://search.worldcat.org/oclc/318368332>).
6. Jones 1997, pp. 229–30.
7. Kleene 1952, pp. 352–3.
8. Cutland, Nigel J. (1980). *Computability: An Introduction to Recursive Function Theory* (<https://books.google.com/books?id=wAstOUE36kcC>). Cambridge University Press. p. 204 (<https://books.google.com/books?id=wAstOUE36kcC&pg=PA204>). doi:10.1017/cbo9781139171496 (<https://doi.org/10.1017%2Fcbo9781139171496>). ISBN 9781139935609. OCLC 488175597 (<https://search.worldcat.org/oclc/488175597>). Retrieved 6 May 2020.
9. Jones 1997.
10. Cutland, Nigel. *Computability: an introduction to recursive function theory*.

11. Barendregt, Henk; Terwijn, Sebastiaan A. (2019). "Fixed point theorems for precomplete numberings" (<http://www.sciencedirect.com/science/article/pii/S016800721930048X>). *Annals of Pure and Applied Logic*. **170** (10): 1151–1161. doi:10.1016/j.apal.2019.04.013 (<https://doi.org/10.1016%2Fj.apal.2019.04.013>). hdl:2066/205967 (<https://hdl.handle.net/2066%2F205967>). ISSN 0168-0072 (<https://search.worldcat.org/issn/0168-0072>). S2CID 52289429 (<https://api.semanticscholar.org/CorpusID:52289429>). Retrieved 6 May 2020. p. 1151.
12. See Ershov 1999, §4.14 for a survey in English.

Further reading

- Jockusch, C. G.; Lerman, M.; Soare, R.I.; Solovay, R.M. (1989). "Recursively enumerable sets modulo iterated jumps and extensions of Arslanov's completeness criterion". *The Journal of Symbolic Logic*. **54** (4): 1288–1323. doi:10.1017/S0022481200041104 (<https://doi.org/10.1017%2FS0022481200041104>). ISSN 0022-4812 (<https://search.worldcat.org/issn/0022-4812>). JSTOR 2274816 (<https://www.jstor.org/stable/2274816>). S2CID 32203705 (<https://api.semanticscholar.org/CorpusID:32203705>).

External links

- "Recursive Functions" (<https://plato.stanford.edu/entries/recursive-functions/>) entry by Piergiorgio Odifreddi in the *Stanford Encyclopedia of Philosophy*, 2012.

This article is issued from [Wikipedia](#). The text is available under [Creative Commons Attribution-Share Alike 4.0](#) unless otherwise noted. Additional terms may apply for the media files.